In God we trust

نام و نام خانوادگی : مهدیس میرزایی

شماره دانشجویی : ۸۱۰۱۰۲۵۵۰

شماره پروژه : پروژه دوم درس

قسمت اول: پیاده‌سازی خط فرمان

در این قسمت از پروژه طبق صورت پروژه به پیاده‌سازی خط فرمان ساده پرداختیم و در آن دستورات گفته شده را پیاده‌سازی کردیم. در این برنامه طبق موارد گفته شده اصول زیر رعایت می‌شود.

۱. مدیریت ورودی: برنامه در یک حلقه بی‌نهایت منتظر دستورات کاربر می‌ماند و با دریافت دستور `exit` خاتمه می‌یابد.

۲. پایپ‌های بی‌نام (Unnamed Pipe) : برای دستوراتی که با نماد `|` جدا شده‌اند ، از فراخوانی سیستمی `pipe()` استفاده شد. با استفاده از `fork()` دو پردازه فرزند ایجاد شد که خروجی اولی به ورودی دومی متصل می‌شود.

۳. پایپ‌های نام‌دار (Named Pipe): نماد `np` به عنوان یک فایل FIFO ثابت در نظر گرفته شد. در صورت مشاهده `< np` و `np >`، برنامه فایل مربوطه را با `mkfifo` ایجاد کرده (در صورت عدم وجود) و ورودی/خروجی استاندارد را با `dup2` به آن فایل هدایت می‌کند.

۴. دستورات داخلی: سه دستور `print_n` ، `echo` و `len` طبق مشخصات خواسته شده پیاده‌سازی شدند.

تست دستورهای گفته شده :

```
❯ make && ./main
g++ -Wall -std=c++17 main.cpp -o main
Welcome. Type 'exit' to quit.
$ print_n 3 | echo | len | echo
19
$ print_n 2 > np
$ print_n 1 > np
$ echo < np
hello, hello
$ echo < np | len
5
$ exit
```

چون به صورت fifo است پس وقتی دستور `echo < np` استفاده شده است صرفا همان اولی را برمی‌گرداند یعنی چون ابتدا ۲ تا hello وارد شده بود پس آن را چاپ می‌کند.

قسمت دوم : تحلیل و مقایسه عملکرد انواع Pipe

بخش اول : سوالات تشریحی

- توضیح دهید که در سیستم عامل لینوکس، انتقال داده بین فرایندها از طریق پایپهای نامدار و بینام چگونه انجام میشود و هر نوع پایپ از چه مکانیزمی بدین منظور استفاده میکند؟ تفاوت اصلی در نحوهٔ ایجاد، محل نگهداری داده‌ها، و نحوهٔ اتصال فرایندها چیست؟

Unnamed Pipe : در لینوکس، این پایپ‌ها با فراخوانی `pipe` ایجاد می‌شوند و تنها یک بافر در Kernel Memory هستند. این پایپ‌ها هیچ نامی در فایل‌سیستم ندارند و صرفاً از طریق File Descriptors که بین پردازه پدر و فرزند به ارث می‌رسند، قابل دسترسی هستند. داده‌ها مستقیماً از حافظه یک پردازه به بافر کرنل و سپس به پردازه دیگر منتقل می‌شوند.

Named Pipe/FIFO : با دستور `mkfifo` ایجاد می‌شود و به صورت یک فایل ویژه در File System ذخیره می‌شود. مکانیزم انتقال داده همچنان از طریق حافظه کرنل است، اما Rendezvous point یک مسیر در دایرکتوری سیستم است.

تفاوت اصلی در **نحوه اتصال** و **طول عمر** است. پایپ بی‌نام موقتی است و با بسته شدن پردازه‌ها از بین می‌رود و نیاز به خویشاوندی پردازه‌ها دارد. پایپ نام‌دار مانند یک فایل روی دیسک می‌ماند (تا زمانی که حذف شود) و هر پردازه‌ای با دسترسی مناسب می‌تواند به آن وصل شود.

- ویژگی های عملکردی پایپ‌های نامدار و بی‌نام را مقایسه کنید. وجود یا تفاوت در مقدار کدام عوامل در سطح سیستم مانند زمان بهره‌وری از CPU، فراخوانی‌های سیستمی، دسترسی به فایل‌سیستم و...، باعث تفاوت در عملکرد آنها میشود و چگونه؟

  پایپ‌های نام‌دار معمولاً سربار بیشتری نسبت به پایپ‌های بی‌نام دارند. دلایل این تفاوت عبارتند از:

  1. **عملیات فایل‌سیستم:** استفاده از پایپ نام‌دار نیازمند ایجاد فایل, بررسی Permissions و `unlink` است. این عملیات‌ها درگیر متادیتاهای دیسک می‌شوند.
  2. **System Time :** به دلیل درگیری بیشتر کرنل برای مدیریت فایل در دایرکتوری، زمان مصرف شده در حالت Kernel Mode برای پایپ‌های نام‌دار بیشتر است.
  3. **کارایی:** پایپ‌های بی‌نام تماماً در حافظه مدیریت می‌شوند و هیچ سربار برای تعامل با ساختار فایل‌سیستم ندارند، بنابراین سریع‌تر هستند.

- در چه شرایطی استفاده از پایپ نامدار نسبت به پایپ بی‌نام مناسب‌تر است و یا برعکس؟ مصالحه میان انعطاف‌پذیری، همگام‌سازی، طول عمر پایپ و عملکرد آن را در استفاده از این دو نوع پایپ به اختصار توضیح دهید.

  استفاده از پایپ بی‌نام: زمانی مناسب است که پردازه‌ها مرتبط هستند (پدر-فرزند) و ارتباط یک‌بار مصرف و خطی است (مانند دستورات شل `ls|grep`). مزیت آن سرعت بالا و پاک‌سازی خودکار است.

  استفاده از پایپ نام‌دار: زمانی استفاده می‌شود که پردازه‌ها هیچ ارتباط خویشاوندی ندارند (مثلاً دو ترمینال جداگانه) یا نیاز است کانال ارتباطی برای مدت طولانی باقی بماند.

  Trade-off: در پایپ نام‌دار، ما راحتی در اتصال پردازه‌های نامرتبط را به دست می‌آوریم با از دست دادن سرعت و مسئولیت synchronization و unlink بر عهده برنامه‌نویس است

بخش دوم : مقایسه عملی

```
mhieir@Linux:~/Uni/Operating system/CA/Operating-System/CA2$ chmod +x named.sh unnamed.sh
benchmark.sh
mhieir@Linux:~/Uni/Operating system/CA/Operating-System/CA2$ ./benchmark.sh named.sh
results_named.txt
Running named.sh 10 times and collecting statistics...
Run 1...
Run 2...
Run 3...
Run 4...
Run 5...
Run 6...
Run 7...
Run 8...
Run 9...
Run 10...
Averages written to results_named.txt
mhieir@Linux:~/Uni/Operating system/CA/Operating-System/CA2$ ./benchmark.sh unnamed.sh
results_unnamed.txt
Running unnamed.sh 10 times and collecting statistics...
Run 1...
Run 2...
Run 3...
Run 4...
Run 5...
Run 6...
Run 7...
Run 8...
Run 9...
Run 10...
Averages written to results_unnamed.txt
mhieir@Linux:~/Uni/Operating system/CA/Operating-System/CA2$ cat resulted_named.txt
cat: resulted_named.txt: No such file or directory
mhieir@Linux:~/Uni/Operating system/CA/Operating-System/CA2$ cat results_named.txt
Command: named.sh
Average User time (seconds): 5.235
Average System time (seconds): 17.706
Average CPU %: 85.00
Average Elapsed time (seconds): 26.804
Average File system inputs: 0.000
Average File system outputs: 0.000
mhieir@Linux:~/Uni/Operating system/CA/Operating-System/CA2$ cat results_unnamed.txt
Command: unnamed.sh
Average User time (seconds): 3.643
Average System time (seconds): 14.626
Average CPU %: 118.30
Average Elapsed time (seconds): 15.408
Average File system inputs: 0.000
Average File system outputs: 0.000
```

طبق خروجی میتوان متوجه شد در بی‌نام زمان کمتر بوده ولی cpu بیشتر بوده است و در نامدار شرایط عکس وجود دارد.

مقایسه زمان اجرا : پایپ بی‌نام (۱۵.۴ ثانیه) حدود ۷۵٪ سریع‌تر از پایپ نام‌دار (۲۶.۸ ثانیه) عمل کرد. System Time در پایپ نام‌دار بیشتر بود (۱۷.۷ ثانیه). علت کندی پایپ نام‌دار ناشی از Filesystem Overhead است. در سناریوی نام‌دار، سیستم‌عامل در هر بار اجرای حلقه مجبور به `mkfifo`، بررسی مجوزها و `unlink` فایل بود که منجر به درگیری بیشتر کرنل و افزایش زمان کل شد. در مقابل، پایپ بی‌نام تمام عملیات را در حافظه و بدون درگیر کردن متادیتای دیسک انجام داد.

مقایسه مصرف CPU : بنچمارک نشان داد که میانگین مصرف CPU در پایپ بی‌نام ۱۱۸٪ و در پایپ نام‌دار ۸۵٪ بوده است. علت عدد ۱۱۸٪ در پایپ بی‌نام نشان‌دهنده Parallelism موثر است؛ به این معنی که پردازه‌های تولیدکننده (dd) و مصرف‌کننده (cat) همزمان روی هسته‌های مختلف CPU اجرا شده‌اند و از ظرفیت سخت‌افزار حداکثر استفاده را کرده‌اند اما در پایپ نام‌دار، پردازنده زمان قابل‌توجهی را در حالت انتظار (I/O Wait) سپری کرده است. در لحظاتی که سیستم منتظر تکمیل ساخت یا حذف فایل در فایل‌سیستم است، CPU بیکار می‌ماند و این باعث کاهش میانگین درصد استفاده از پردازنده شده است.